

GhostPredict (GPA)

Documentação Técnica Completa

Compressão lossless com prior embutido no codec para micro-payloads de IoT

Algoritmo com trechos de código comentados, base matemática, geração do primer, reprodução integral do experimento e guia de reimplementação

Matheus Araújo

araujomatheuscarv@gmail.com

Versão do documento: 2.1

Implementação de referência: ghost_core.rs e main.rs (Rust)

Repositório: <https://github.com/matheusaraujoc/GPA>

Sumário

Sumario: no Word, clique aqui e tecle F9 para gerar os numeros de pagina.

1. Resumo executivo

O GhostPredict, abreviado GPA, é um compressor de dados sem perdas projetado para um regime específico e mal atendido pelos compressores de uso geral: mensagens muito curtas, da ordem de dezenas de bytes, como as que predominam na telemetria de Internet das Coisas. Nesse regime, formatos como gzip, zstd e brotli costumam aumentar o tamanho do dado, pois o custo fixo de cabeçalho e enquadramento supera a economia possível. O GPA ataca esse problema com três decisões de projeto que se reforçam: um formato sem cabeçalho, em que o arquivo é apenas o fluxo aritmético; um prior de domínio embutido no codec, e não no arquivo, que aquece o modelo estatístico e serve de dicionário; e uma garantia de nunca expandir a entrada além de uma margem desprezível.

Este documento foi escrito para ser autossuficiente. Ele apresenta a base matemática, descreve cada estágio do algoritmo acompanhado do trecho de código correspondente da implementação de referência em Rust, explica como o primer é gerado e usado, e detalha o procedimento experimental com precisão suficiente para que um terceiro reproduza os resultados e reimplemente o sistema sem ambiguidade. As convenções de leitura são simples: os blocos sombreados em fonte monoespaçada são trechos verbatim do código-fonte; as equações aparecem centralizadas; e as tabelas trazem números medidos, não estimados.

2. Introdução e motivação

2.1. O problema dos micro-payloads

Sensores, rastreadores e gateways de IoT emitem mensagens curtas e frequentes. Um registro típico de telemetria tem entre trinta e cem bytes e viaja por protocolos como MQTT e CoAP, muitas vezes sobre enlaces de baixa potência como LoRaWAN, NB-IoT, Sigfox ou BLE. Nesses enlaces cada byte transmitido custa energia da bateria, tempo de ocupação do canal e, em algumas faixas, esbarra em limites regulatórios de ciclo de trabalho. Reduzir o número de bytes por mensagem tem efeito direto sobre autonomia, custo e capacidade do sistema.

Os compressores de uso geral foram otimizados para arquivos, não para mensagens. Eles inserem cabeçalhos, somas de verificação e estruturas de bloco cujo custo é constante e independe do tamanho da entrada. Quando a entrada tem poucas dezenas de bytes, esse custo fixo passa a dominar e o resultado comprimido sai maior do que o original. Esse efeito é demonstrado quantitativamente na Seção 11.

2.2. A proposta e as contribuições

O GPA parte da hipótese de que um compressor desenhado para o nicho pode inverter esse quadro. A ideia central é que o conhecimento prévio sobre o formato das mensagens não precisa viajar com cada mensagem: ele pode residir uma única vez no codec, distribuído junto ao firmware, e ser usado de forma idêntica pelo compressor e pelo descompressor. As contribuições deste trabalho são o algoritmo e o formato sem cabeçalho, o mecanismo de prior embutido, a garantia de não expansão, e uma avaliação em onze conjuntos de dados reais sob protocolo por mensagem com separação entre treino e teste.

3. Fundamentos matemáticos

3.1. Entropia e o limite de Shannon

A teoria da informação fixa o piso de qualquer compressor sem perdas. Para uma fonte que emite símbolos x com probabilidade $p(x)$, a entropia de Shannon é

$$H(X) = - \sum_x p(x) \cdot \log_2 p(x) \quad [\text{bits por símbolo}].$$

Nenhum código univocamente decodificável usa, em média, menos do que $H(X)$ bits por símbolo. O comprimento ideal do código de um símbolo de probabilidade p é $\ell = -\log_2 p$. Todo compressor estatístico se resume a estimar $p(x)$ bem e a gastar exatamente esse número de bits, tarefa para a qual a codificação aritmética é ótima.

3.2. Por que micro-payloads incham

Seja c o custo fixo em bytes de um formato e r a razão de compressão alcançável sobre o conteúdo. O tamanho de saída de uma entrada de n bytes é $\text{saída}(n) \approx c + r \cdot n$. Para haver compressão é preciso $c + r \cdot n < n$, isto é,

$$n > c / (1 - r).$$

Com um custo fixo de oito a vinte bytes, mensagens abaixo de algumas dezenas de bytes estão matematicamente fadadas a inflar, independentemente da qualidade do modelo. A resposta do GPA é levar c a zero, eliminando o cabeçalho, e reduzir r por meio do prior.

3.3. Codificação aritmética inteira

A codificação aritmética representa uma sequência inteira de símbolos por um único número em um subintervalo de $[0, 1)$, estreitado a cada símbolo na proporção de sua probabilidade. O GPA usa a variante inteira de 32 bits de Witten, Neal e Cleary, que evita ponto flutuante e é exatamente reproduzível. O estado é o par low e high , inicialmente 0 e $2^{32} - 1$. Dado o modelo de um símbolo, com acumulados c_low e c_high sobre um total T ,

$$\text{range} = \text{high} - \text{low} + 1$$

$$\text{high} \leftarrow \text{low} + \lfloor \text{range} \cdot c_high / T \rfloor - 1, \quad \text{low} \leftarrow \text{low} + \lfloor \text{range} \cdot c_low / T \rfloor.$$

Em seguida o intervalo é renormalizado, emitindo bits à medida que low e high convergem, com tratamento explícito do caso de underflow por meio de bits pendentes. A Listagem 1 mostra a rotina exata da implementação, com as constantes $\text{HALF} = 2^{31}$, $Q1 = 2^{30}$ e $Q3 = 3 \cdot 2^{30}$.

Listagem 1. Codificador aritmético: estreitamento do intervalo e renormalização (ghost_core.rs).

```
pub fn encode(&mut self, low_count: u16, high_count: u16, total_count: u16, writer: &mut
BitWriter) {
    let range = (self.high as u64) - (self.low as u64) + 1;
    self.high = self.low + (((range * (high_count as u64)) / (total_count as u64)) as u32) -
1;
    self.low = self.low + (((range * (low_count as u64)) / (total_count as u64)) as u32);
    loop {
        if self.high < HALF { // bit de alta ordem = 0
            self.emit_bit(0, writer);
        } else if self.low >= HALF { // bit de alta ordem = 1
            self.emit_bit(1, writer);
            self.low -= HALF; self.high -= HALF;
        } else if self.low >= Q1 && self.high < Q3 { // underflow: convergencia em torno do
meio
            self.pending_bits += 1;
            self.low -= Q1; self.high -= Q1;
```

```

    } else { break; }
    self.low = (self.low << 1) & MAX_VALUE; // desloca, entra 0 no low
    self.high = ((self.high << 1) | 1) & MAX_VALUE; // desloca, entra 1 no high
  }
}

```

A função `emit_bit` resolve o underflow: ao emitir um bit `b`, emite em seguida os `pending_bits` acumulados com o valor oposto, técnica padrão que preserva o código livre de prefixo. A terminação fixa o último bit conforme o quadrante de `low`, como na Listagem 2.

Listagem 2. Emissão de bits com pendências e terminação do fluxo.

```

fn emit_bit(&mut self, bit: u32, writer: &mut BitWriter) {
  writer.write_bit(bit);
  for _ in 0..self.pending_bits { writer.write_bit(bit ^ 1); }
  self.pending_bits = 0;
}
pub fn finish(&mut self, writer: &mut BitWriter) {
  self.pending_bits += 1;
  if self.low < Q1 { self.emit_bit(0, writer); } else { self.emit_bit(1, writer); }
  writer.flush();
}

```

A decodificação é simétrica. O decodificador lê os primeiros 32 bits em uma variável `value` e, para cada símbolo, calcula um alvo que indica em que faixa da tabela acumulada o valor caiu, conforme a Listagem 3. O símbolo é localizado por busca binária na tabela acumulada e, em seguida, aplica-se a mesma renormalização, agora alimentando `value` com novos bits.

Listagem 3. Decodificador: cálculo do alvo e renormalização.

```

pub fn get_target(&self, total_count: u16) -> u16 {
  let range = (self.high as u64) - (self.low as u64) + 1;
  (((self.value as u64) - (self.low as u64) + 1) * (total_count as u64) - 1) / range as
  u16
}
pub fn decode(&mut self, low_count: u16, high_count: u16, total_count: u16, reader: &mut
BitReader) {
  let range = (self.high as u64) - (self.low as u64) + 1;
  self.high = self.low + (((range * (high_count as u64)) / (total_count as u64)) as u32) -
  1;
  self.low = self.low + (((range * (low_count as u64)) / (total_count as u64)) as u32);
  loop {
    if self.high < HALF { /* pass */ }
    else if self.low >= HALF { self.low -= HALF; self.high -= HALF; self.value -= HALF; }
    else if self.low >= Q1 && self.high < Q3 { self.low -= Q1; self.high -= Q1; self.value
    -= Q1; }
    else { break; }
    self.low = (self.low << 1) & MAX_VALUE;
    self.high = ((self.high << 1) | 1) & MAX_VALUE;
    self.value = ((self.value << 1) | reader.read_bit()) & MAX_VALUE;
  }
}

```

Como todas as operações são inteiras e deterministas, a saída do decodificador casa bit a bit com a entrada do codificador, em qualquer arquitetura.

3.4. Modelagem de contexto e PPM com exclusão

A qualidade da compressão depende da estimativa de $p(x)$. O GPA usa um modelo PPM de ordem 2 sobre nibbles: a previsão é condicionada nos últimos dois nibbles, equivalentes a um byte de contexto, gerando 256 contextos. Cada contexto guarda contagens por símbolo. Quando o símbolo já foi visto no contexto, ele é codificado diretamente; quando não, emite-se um escape e recorre-se ao modelo de ordem 0. A massa de escape segue o método A do PPM, com peso fixo igual a um, de modo que em um contexto com total T tem-se $P(\text{escape}) = 1/(T+1)$ e $P(s) = \text{contagem}(s)/(T+1)$. A Listagem 4 mostra a decisão de codificação, incluindo a exclusão: ao cair no modelo de ordem 0, os

símbolos já presentes no contexto são removidos da distribuição, pois se um deles fosse correto não teria havido escape.

Listagem 4. Decisão de codificação do símbolo principal (encode_token), com escape e exclusão.

```
let edges = &self.graph[self.current_node];
let t = self.graph_totals[self.current_node];
let edge_x = edges[symbol];
if t == 0 {
    // contexto inedito -> ordem 0 direto
    coder.encode(self.o0_cum[symbol], self.o0_cum[symbol + 1], self.o0_cum[ALPHABET_SIZE],
writer);
} else if edge_x > 0 {
    // simbolo visto no contexto
    let adjusted = t + 1;
    // +1 reserva a massa do escape
    let ecum = &self.graph_cum[self.current_node];
    coder.encode(ecum[symbol], ecum[symbol + 1], adjusted, writer);
} else {
    // escape + ordem 0 com exclusao
    let adjusted = t + 1;
    coder.encode(t, t + 1, adjusted, writer); // emite o escape em [t, t+1)
    let mut masked_total = self.o0_cum[ALPHABET_SIZE];
    let mut masked_low = self.o0_cum[symbol];
    for j in 0..ALPHABET_SIZE {
        if edges[j] > 0 {
            // exclui simbolos ja vistos no contexto
            let c = self.o0_counts[j];
            masked_total -= c;
            if j < symbol { masked_low -= c; }
        }
    }
    let masked_high = masked_low + self.o0_counts[symbol];
    coder.encode(masked_low, masked_high, masked_total, writer);
}
```

Após codificar, o modelo é atualizado e, quando o total de um contexto atinge 2048, todas as contagens não nulas são divididas por dois, preservando ao menos uma unidade. Esse reescalonamento mantém as contagens dentro de um inteiro de 16 bits e dá ao modelo memória de curto prazo, favorecendo eventos recentes. A Listagem 5 mostra a atualização e o reescalonamento.

Listagem 5. Aprendizado e reescalonamento do contexto.

```
self.graph[self.current_node][symbol] = edge_x + 1;
self.graph_totals[self.current_node] = t + 1;
if self.graph_totals[self.current_node] >= 2048 {
    let mut sum_ctx = 0;
    for s in 0..ALPHABET_SIZE {
        if self.graph[self.current_node][s] > 0 {
            self.graph[self.current_node][s] = (self.graph[self.current_node][s] >> 1) | 1;
            sum_ctx += self.graph[self.current_node][s];
        }
    }
    self.graph_totals[self.current_node] = sum_ctx;
}
```

4. Arquitetura do algoritmo

4.1. Visão geral do pipeline

O núcleo transforma a sequência de bytes em um único fluxo de bits por quatro estágios encadeados, processados símbolo a símbolo, sem materializar estruturas proporcionais à entrada.

```
bytes -> LZ77 (janela deslizante): literais e referencias (distancia, comprimento)
-> fluxo de tokens: nibbles, simbolo de match, EOF
-> PPM ordem-2 sobre nibbles + modelos de distancia e comprimento
-> codificador aritmetico inteiro de 32 bits
-> .gpa (apenas o fluxo de bits, sem cabecalho)
```

4.2. Alfabeto, tokens e contexto

O alfabeto tem 18 símbolos: 0 a 15 são nibbles, 16 é o EOF e 17 indica um match. Cada byte literal vira dois nibbles, o de ordem alta primeiro. O contexto de ordem 2 é guardado em uma variável e avança apenas em nibbles literais, pela operação

$$\text{contexto} \leftarrow ((\text{contexto} \& 0x0F) \ll 4) \mid \text{símbolo},$$

de modo que match e EOF não perturbam a vizinhança de nibbles. As constantes que definem o alfabeto e o codificador são as da Listagem 6.

Listagem 6. Constantes do alfabeto, do codificador e da flag de modo.

```
pub const MAX_VALUE: u32 = 0xFFFFFFFF; pub const HALF: u32 = 0x80000000;
pub const Q1: u32 = 0x40000000; pub const Q3: u32 = 0xC0000000;
pub const ALPHABET_SIZE: usize = 18; // 16 nibbles + EOF + MATCH
pub const EOF_SYMBOL: usize = 16; pub const MATCH_SYMBOL: usize = 17;
pub const FLAG_TOTAL: u16 = 64; pub const FLAG_STORED_LOW: u16 = 63; // flag enviesada 1/64
```

4.3. Estágio LZ77

A busca usa uma tabela head indexada pelo hash de três bytes e um vetor prev que encadeia posições de mesmo hash. A Listagem 7 mostra o buscador: ele percorre a cadeia até 128 candidatos dentro da janela, aplica um filtro rápido comparando primeiro o byte na posição do melhor comprimento já encontrado, e retorna o casamento mais longo, entre 4 e 67 bytes.

Listagem 7. Buscador de casamentos do LZ77 (find_match).

```
fn find_match(&self, raw: &[u8], i: usize, n: usize) -> (usize, usize) {
    if i + LZ_MIN_MATCH > n { return (0, 0); }
    let hv = self.hash(raw, i);
    let mut cand = self.head[hv] as isize;
    let (mut best_len, mut best_dist, mut attempts) = (0usize, 0usize, 0usize);
    let limit = min(LZ_MAX_MATCH, n - i);
    while cand >= 0 && (i - cand as usize) <= self.window && attempts < LZ_MAX_CHAIN {
        let c_pos = cand as usize;
        if best_len > 0 {
            if i + best_len >= n { break; }
            if raw[c_pos + best_len] != raw[i + best_len] { // filtro rapido
                cand = self.prev[c_pos & self.win_mask] as isize; attempts += 1; continue;
            }
        }
        let mut l = 0;
        while l < limit && raw[c_pos + l] == raw[i + l] { l += 1; }
        if l > best_len && l >= LZ_MIN_MATCH {
            best_len = l; best_dist = i - c_pos;
            if l >= limit { break; }
        }
        cand = self.prev[c_pos & self.win_mask] as isize; attempts += 1;
    }
    (best_len, best_dist)
}
```

O parser aplica casamento preguiçoso: antes de fixar um match na posição corrente, verifica se a posição seguinte oferece um casamento mais longo; em caso afirmativo, emite o byte atual como dois nibbles e adia. A Listagem 8 mostra o laço principal do parser em modo de fluxo, que emite cada token por retorno de chamada, sem materializar o vetor de tokens.

Listagem 8. Laço do parser em fluxo, com casamento preguiçoso (parse_streaming).

```
let (cur_len, cur_dist) = self.find_match(raw, i, n);
self.insert_hash(raw, i, n);
if cur_len >= LZ_MIN_MATCH {
    if i + 1 < n && cur_len < LZ_MAX_MATCH { // lazy matching
        let (next_len, _) = self.find_match(raw, i + 1, n);
        if next_len > cur_len {
```

```

        let b = raw[i];
        emit(Token::Nibble((b >> 4) & 0x0F)); emit(Token::Nibble(b & 0x0F));
        i += 1; continue;
    }
}
emit(Token::Match { dist: cur_dist as u16, len: cur_len as u8 });
for j in 1..cur_len { self.insert_hash(raw, i + j, n); }
i += cur_len;
} else {
    let b = raw[i];
    emit(Token::Nibble((b >> 4) & 0x0F)); emit(Token::Nibble(b & 0x0F));
    i += 1;
}
}

```

4.4. Modelos de distância e comprimento

A distância de um match é codificada por um código de 19 valores: os três primeiros representam, por recência, as três últimas distâncias, mantidas por uma política de mover-para-frente; os demais correspondem a 16 faixas exponenciais, com a posição exata dentro da faixa enviada como bits extras equiprováveis. O comprimento segue esquema análogo, com um valor para repetir o último comprimento e sete faixas. As bases e os bits extras estão na Tabela A.2. A Listagem 9 mostra a escolha do código de distância e a emissão dos bits extras.

Listagem 9. Codificação da distância: recência, faixa e bits extras.

```

let dist_code = if dist == self.last_offsets[0] { 0 }
                else if dist == self.last_offsets[1] { 1 }
                else if dist == self.last_offsets[2] { 2 }
                else { 3 + bucket_dist(dist) };
coder.encode(self.dist_code_cum[dist_code], self.dist_code_cum[dist_code + 1],
             self.dist_code_cum[DIST_CODE_SIZE], writer);
if dist_code >= 3 {
    let db = dist_code - 3;
    let extra = dist - DIST_BASE[db];
    for b in (0..DIST_EXTRA[db]).rev() { coder.encode_bit(((extra >> b) & 1) as u16,
writer); }
}
mtf_offsets(&mut self.last_offsets, dist);

```

5. O prior: geração e funcionamento

Esta seção responde, em detalhe, como o primer é gerado, o que ele contém, como é embutido e como aquece o modelo. É a contribuição central do trabalho e o ponto que diferencia o GPA dos compressores gerais.

5.1. O que é o primer

O primer é um corpus bruto de mensagens representativas do domínio. Não tem formato especial: é apenas a concatenação de amostras, uma por linha. O motor o embute em tempo de compilação e o trata como bytes. A constante PRIMER e o cabeçalho de inclusão são os da Listagem 10.

Listagem 10. Inclusão do primer no binário em tempo de compilação.

```
pub const PRIMER: &[u8] = include_bytes!("primer.bin");
```

Como o primer é compilado tanto no compressor quanto no descompressor, ele nunca entra no arquivo .gpa. Os dois lados o conhecem de forma idêntica, então o arquivo permanece livre de dicionário. Trocar o primer significa substituir o arquivo primer.bin e recompilar.

5.2. Dimensionamento

O primer cumpre dois papéis com requisitos diferentes de tamanho. Para o aquecimento do modelo PPM, todo o corpus conta: quanto mais amostras, melhor a estimativa inicial das probabilidades. Para

o dicionário LZ, porém, apenas os últimos bytes importam, pois a janela deslizante tem 32 KB; cadeias além desse limite não são alcançáveis como referências. Por isso o gerador, por padrão, mantém os últimos 32768 bytes do corpus, valor igual à janela de fluxo. Um primer maior aquece mais o PPM, mas torna o aquecimento mais lento por mensagem e aumenta o tamanho do binário.

5.3. O gerador de primer

O script `gen_primer.py` constrói o primer em três modos. O modo a partir de dataset, usado na validação experimental, lê um arquivo com um registro por linha, embaralha com semente fixa, reserva as primeiras duzentas linhas como teste, que nunca entram no primer, e usa as seguintes como treino. O ponto crucial dessa separação é que o primer é construído apenas com dados de treino, o que garante que a avaliação meça generalização, e não memorização. A Listagem 11 mostra o núcleo do gerador.

Listagem 11. Núcleo do gerador de primer (`gen_primer.py`).

```
def cap_tail(corpus: bytes, cap: int) -> bytes:
    # Mantem os ULTIMOS cap bytes: sao os que a janela LZ enxerga como dicionario.
    return corpus[-cap:] if cap and len(corpus) > cap else corpus

def from_dataset(path, seed, skip_header, cap, heldout):
    lines = [ln.rstrip("\n").rstrip("\r") for ln in open(path, encoding="utf-8",
errors="replace")]
    if skip_header and lines: lines = lines[1:]
    lines = [x for x in lines if x.strip()]
    random.Random(seed).shuffle(lines)
    train = lines[N_TEST:N_TEST + heldout]      # as primeiras N_TEST sao TESTE (held-out)
    corpus = ("\n".join(train) + "\n").encode("utf-8", "replace")
    return cap_tail(corpus, cap), len(train)
```

A invocação típica para um domínio, e a recompilação subsequente, são:

```
python gen_primer.py --from-dataset datasets/.../loop_1.csv --skip-header --cap 32768
rustc +stable-x86_64-pc-windows-gnu -C opt-level=3 main.rs -o main.exe
```

O gerador também oferece um modo sintético, que produz de forma determinística um corpus genérico de texto curto estruturado, com palavras, pares chave igual valor, URLs, endereços e objetos JSON, útil quando não há um corpus de domínio disponível.

5.4. Como o primer aquece o modelo

Antes de processar a mensagem, ambos os lados percorrem o primer e atualizam o modelo sem emitir nem ler bits. A rotina de aquecimento parseia o primer com o mesmo LZ77 e aplica a cada token a mesma atualização da codificação, exceto o EOF. A Listagem 12 mostra esse aquecimento.

Listagem 12. Aquecimento do modelo pelo primer (`prime_model`).

```
fn prime_model(&mut self, primer: &[u8]) {
    let mut lz = LZ77::new(LZ_WINDOW, LZ_HASH_SIZE);
    let mut toks: Vec<Token> = Vec::new();
    lz.parse_streaming(primer, |t| toks.push(t));
    for t in toks {
        if !matches!(t, Token::EOF) { self.learn_token(t); } // mesma atualizacao, sem
codificar
    }
}
```

Na compressão com prior, a mensagem é concatenada após o primer e o parser de dicionário pré-carrega a tabela de hash com as posições do primer, emitindo apenas os tokens da mensagem. Assim, um match da mensagem pode apontar para uma cadeia do primer. A Listagem 13 mostra esse caminho.

Listagem 13. Compressão com prior: aquecimento e dicionário (`compress_primed`).

```
pub fn compress_primed(&mut self, raw: &[u8], primer: &[u8]) -> Vec<u8> {
```

```

    self.prime_model(primer); // aquece PPM + dist/len
    let mut combined = Vec::with_capacity(primer.len() + raw.len());
    combined.extend_from_slice(primer); combined.extend_from_slice(raw);
    let mut writer = BitWriter::new(); let mut coder = ArithmeticCoder::new();
    let mut lz = LZ77::new(LZ_WINDOW, LZ_HASH_SIZE);
    lz.parse_with_dict(&combined, primer.len(), // emite so os tokens da
mensagem
    |tok| self.encode_token(tok, &mut coder, &mut writer));
    coder.finish(&mut writer);
    writer.bytes
}

```

O descompressor executa `prime_model` com o mesmo `primer`, reconstrói a mensagem em um buffer prefixado pelo `primer`, para que os matches no dicionário funcionem, e ao final descarta o prefixo. Como o aquecimento é idêntico nos dois lados e usa apenas aritmética inteira, a reconstrução é exata.

6. Modos de operação

O GPA expõe três pares de modos. A escolha do modo é explícita; a seleção de perfil de janela e a garantia de não expansão operam de forma automática dentro do modo normal.

6.1. Modo normal e a garantia de não expansão (c, d)

O modo normal comprime um arquivo inteiro. Ele escreve a flag enviesada, seleciona o perfil de janela pelo tamanho da entrada e, se o resultado exceder o original, recorre ao modo de armazenamento. A flag ocupa um total de 64: o modo normal recebe $[0, 63)$ e o modo de armazenamento recebe $[63, 64)$, uma fração de um sobre sessenta e quatro. O custo é $-\log_2(63/64) \approx 0,0227$ bit no caso comprimido e $-\log_2(1/64) = 6$ bits no armazenado. A Listagem 14 mostra o caminho completo.

Listagem 14. Modo normal com flag enviesada, perfil automático e fallback de armazenamento.

```

pub fn compress_stream(&mut self, raw: &[u8]) -> Vec<u8> {
    let mut writer = BitWriter::new(); let mut coder = ArithmeticCoder::new();
    coder.encode(0, FLAG_STORED_LOW, FLAG_TOTAL, &mut writer); // is_stored = 0 (~0.02
bit)
    let (window, hash_size) = if raw.len() <= LZ_WINDOW_MICRO {
        (LZ_WINDOW_MICRO, LZ_HASH_MICRO) // perfil micro (<= 4 KB)
    } else { (LZ_WINDOW, LZ_HASH_SIZE); // perfil de fluxo
    let mut lz = LZ77::new(window, hash_size);
    lz.parse_streaming(raw, |tok| self.encode_token(tok, &mut coder, &mut writer));
    coder.finish(&mut writer);
    let comp = writer.bytes;
    if comp.len() <= raw.len() + 1 { return comp; }
    // INFLOU -> modo de armazenamento: flag = 1 + bytes em modelo plano de 257 + EOF
    let mut w2 = BitWriter::new(); let mut c2 = ArithmeticCoder::new();
    c2.encode(FLAG_STORED_LOW, FLAG_TOTAL, FLAG_TOTAL, &mut w2); // is_stored = 1 (6 bits)
    for &b in raw { c2.encode(b as u16, b as u16 + 1, 257, &mut w2); }
    c2.encode(256, 257, 257, &mut w2); // EOF do fluxo plano
    c2.finish(&mut w2);
    let stored = w2.bytes;
    if stored.len() < comp.len() { stored } else { comp }
}

```

No modo de armazenamento cada byte custa $\log_2 257 \approx 8,0056$ bits, acréscimo de cerca de sete centésimos de por cento, somado aos seis bits da flag e à terminação. A expansão máxima fica limitada a $|saída| \leq |entrada| \cdot (1 + 0,0007) + O(1)$, na prática algo como a entrada mais cerca de três bytes.

6.2. Modo com prior (cp, dp e cpf, dpf)

As variantes `cp` e `dp` usam o `primer` compilado; `cpf` e `dpf` recebem o `primer` como arquivo em tempo de execução, o que permite trocar de domínio sem recompilar. As variantes de arquivo foram usadas

na validação para garantir a separação entre treino e teste. O arquivo gerado contém apenas a mensagem.

6.3. Modo de fluxo em blocos (cs, ds)

Para arquivos grandes fora do nicho, o modo de fluxo divide a entrada em blocos de 4 MB e comprime cada bloco de forma independente, com memória de trabalho limitada ao tamanho do bloco. O envelope é o container .gpas descrito na Seção 7.

7. Formato de arquivo

7.1. Arquivo .gpa no modo normal

O arquivo é exclusivamente o fluxo aritmético. O primeiro evento decodificado é a flag, sobre um total de 64. Se o alvo for menor que 63, segue o modo comprimido: uma sucessão de símbolos PPM, em que nibbles remontam bytes, o símbolo de match dispara a leitura de distância e comprimento e a cópia da janela, e o EOF encerra. Se o alvo for 63, segue o modo de armazenamento: bytes por um modelo plano de 257, terminado pelo símbolo 256.

7.2. Arquivo .gpa no modo com prior

É o fluxo aritmético puro da mensagem, sem flag e sem cabeçalho. A decodificação exige o mesmo primer, pois o modelo precisa ser aquecido de forma idêntica e os matches podem referenciar posições do primer.

7.3. Container .gpas no modo de fluxo

Começa com a assinatura de cinco bytes GPAS1. Em seguida, para cada bloco, um inteiro de 32 bits little-endian com o comprimento do bloco, seguido dos bytes do bloco no formato .gpa normal. Um comprimento zero marca o fim.

['G''P''A''S''1']	assinatura (5 bytes)
[u32 LE = L1][bloco .gpa de L1 bytes]	bloco 1
[u32 LE = L2][bloco .gpa de L2 bytes]	bloco 2
...	
[u32 LE = 0]	terminador

8. Implementação

8.1. Organização do código

A implementação está em Rust em dois arquivos. O módulo `ghost_core.rs` contém o motor: codificador e decodificador aritméticos, buscador LZ77, modelos estatísticos e as rotinas de compressão e descompressão dos três modos. É o algoritmo em si; reimplementar o GPA significa portar este arquivo. O arquivo `main.rs` contém a interface de linha de comando, a suíte de conformidade, o benchmark, o alocador instrumentado e o enquadramento do container .gpas. À exceção desse envelope, `main.rs` é ferramenta, não algoritmo.

8.2. Tipos compactos e perfis de memória

As contagens são limitadas a 2048 pelo reescalonamento, o que permite armazená-las em inteiros de 16 bits; o grafo de contextos e suas tabelas acumuladas usam esse tipo, o que reduziu o footprint do decodificador sem alterar a saída. As tabelas do LZ77 são dimensionadas em tempo de execução

conforme o perfil: micro, para entradas de até 4 KB, com janela de 4 KB e hash reduzida; e de fluxo, com janela de 32 KB.

8.3. Medição de memória

A implementação instala um alocador global instrumentado que registra o pico de heap vivo, o que permite medir a memória do algoritmo sem o ruído do tempo de execução nem o conjunto residente do sistema. O subcomando de benchmark usa esse alocador para reportar o footprint de compressão e de descompressão separadamente.

9. Reprodução: build, comandos e conformidade

9.1. Compilação

O motor compila com o compilador Rust, cadeia GNU no Windows, pois o ligador MSVC falha neste projeto. O comando de compilação otimizada é:

```
rustc +stable-x86_64-pc-windows-gnu -C opt-level=3 main.rs -o main.exe
```

9.2. Referência de comandos

Comando	Função
c entrada saída.gpa	Comprimir (flag enviesada, perfil automático, nunca inflar)
d entrada.gpa saída	Descomprimir o modo normal
cp / dp entrada saída	Comprimir / descomprimir com primer embutido
cpf / dpf entrada saída primer	Comprimir / descomprimir com primer em arquivo
cs / ds entrada saída	Fluxo de blocos (RAM limitada)
t	Suíte de conformidade
bench entrada [iterações]	Benchmark em memória (tempo e heap)

9.3. Suíte de conformidade

O comando t verifica, para cada caso, três critérios: round-trip sem perdas, ausência de expansão relevante e igualdade do tamanho gerado com um valor de referência, este último como guarda contra regressões. Os vetores e tamanhos estão na Tabela B.1. A bateria roda em milissegundos e é pré-requisito para qualquer alteração no motor.

9.4. Benchmark e footprint

O comando bench cronometra a compressão e a descompressão em memória, sem I/O de disco, e reporta o pico de heap de cada uma. A saída é uma linha parseável. Os valores medidos para uma mensagem típica e para uma entrada de 64 KB estão na Tabela 8 da Seção 11.

10. Reprodução integral do experimento

Esta seção descreve o experimento com detalhe suficiente para reprodução exata. Todas as etapas usam sementes fixas e ferramentas de versão conhecida.

10.1. Conjuntos de dados e procedência

São onze fluxos de registros curtos de oito fontes públicas. A Tabela 1 lista a procedência. Cada fonte é tratada como um fluxo com um registro por linha; para o GeoLife, descartam-se as seis linhas de cabeçalho de cada arquivo de trajetória.

Tabela 1. Procedência dos conjuntos de dados.

Domínio	Fonte	Origem
MQTT	MQTTEEB-D	Mendeley Data, DOI 10.17632/jfttfjn6tr
AIS	MarineCadastro AIS	marinecadastre.gov/accessais (NOAA e BOEM)
Sensores	Intel Lab Data	db.csail.mit.edu/labdata/labdata.html
Energia	UCI Household Power	UCI ML Repository, DOI 10.24432/C58K54
GPS	GeoLife	Microsoft Research (Zheng et al. 2009)
Logs (4)	Loghub	github.com/logpai/loghub (Zhu et al. 2023)
SMS	SMS Spam Collection	UCI ML Repository, DOI 10.24432/C5CC84
Tweets	Sentiment140	Go, Bhayani e Huang 2009 (Stanford)

10.2. Protocolo por mensagem

Para cada domínio, os registros são lidos, embaralhados com a semente 42 e divididos em teste e treino disjuntos. O teste tem as primeiras mil mensagens; o treino tem as seguintes, até cinco mil. A única exceção são os quatro conjuntos de log do Loghub, que fornecem duas mil linhas por tipo na versão amostrada usada aqui; nesses casos o teste tem mil mensagens e o treino as mil restantes, e as versões completas do Loghub permitiriam ampliar ambos. O treino constrói tanto o primer do GPA, pelos últimos 32 KB do corpus, quanto o dicionário do zstd, treinado pelas mesmas amostras, de modo que ambos recebem a mesma informação prévia. Cada mensagem de teste é então comprimida individualmente por todas as ferramentas. A medição por mensagem, e não por arquivo inteiro, é essencial: medir arquivos inteiros diluiria o custo de partida a frio e esconderia justamente o efeito que o prior corrige. A Listagem 15 mostra a construção do primer e do dicionário no harness.

Listagem 15. Construção do primer e do dicionário a partir do treino (benchmark.py).

```
def build_primer(heldout):
    corpus = ("\n".join(heldout) + "\n").encode("utf-8", "replace")[-PRIMER_CAP:] # ultimos
    32 KB
    p = os.path.join(TMP, "primer.bin"); open(p, "wb").write(corpus); return p

def build_dict(heldout):
    train = os.path.join(TMP, "_train.bin")
    open(train, "wb").write(("n".join(heldout) + "\n").encode("utf-8", "replace"))
    d = os.path.join(TMP, "dict.bin")
    avg = max(16, int(statistics.mean([len(x) for x in heldout])))
    subprocess.run(["zstd", "--train", train, f"-B{avg}", "-o", d, "--maxdict=16384"], ...)
    return d if os.path.exists(d) else None
```

A compressão de cada mensagem com o GPA usa a variante com primer em arquivo, o que evita recompilar o motor a cada domínio:

```
def c_gpa_primed(b, ctx):
    i = os.path.join(TMP, "mp"); o = os.path.join(TMP, "mp.gpa")
    open(i, "wb").write(b)
    run_ok([MAIN, "cpf", i, o, ctx["primer"]])    # comprime com o primer do dominio
    return os.path.getsize(o)
```

10.3. Ferramentas de comparação

Comparam-se dez configurações: o GPA sem prior e com prior; os especialistas de string curta Unishox2 e SMAZ, que têm codebook fixo e nenhum prior de domínio; o gzip nível 9; o zstd nível 19, com e sem dicionário treinado; uma variante enxuta do dicionário do zstd, com as opções --no-check e --no-dictID, que remove a soma de verificação e o identificador de dicionário para isolar a contribuição do modelo da contribuição do enquadramento; o brotli nível 11; e o xz nível 9. As versões foram zstd 1.5.7, xz 5.6.3, gzip 1.13, brotli 1.1.0, e Unishox2 e SMAZ compilados do fonte oficial. As métricas são o tamanho médio, a mediana e o percentil 95 por mensagem, a fração de mensagens expandidas, a fração de vitórias, a vazão e o uso de memória.

Para responder à pergunta sobre a dependência de domínio, executa-se ainda um experimento cruzado: o primer de um domínio é usado para comprimir as mensagens de teste de outro domínio, formando a matriz da Tabela 6. A variante enxuta e o experimento cruzado atendem a uma exigência de rigor: separar o quanto do ganho vem do formato sem cabeçalho do quanto vem do modelo aquecido, e medir o que ocorre quando o prior não casa com o domínio.

10.4. Pré-requisitos

A reprodução exige um pequeno conjunto de ferramentas, todas verificadas pelo próprio benchmark.py antes de iniciar. São elas o compilador Rust com a cadeia GNU; o compilador C, para os especialistas de string curta; os utilitários zstd, xz, gzip e brotli no caminho de busca; e o interpretador Python com a biblioteca psutil, opcional, para a medição de memória. As versões usadas estão no Apêndice D. Os conjuntos de dados, por serem grandes, não acompanham o repositório; suas fontes públicas estão na Tabela 1, e a estrutura de pastas esperada está documentada nas funções de extração do benchmark.py.

10.5. Passo a passo da reprodução

A sequência completa, a partir do repositório com os dados em datasets, é a seguinte. Os comandos de compilação dos especialistas de string curta usam o gcc; os demais são diretos.

```
# 1. compilar o motor (cadeia GNU; o ligador MSVC falha neste projeto)
rustc +stable-x86_64-pc-windows-gnu -C opt-level=3 main.rs -o main.exe

# 2. verificar a corretude (round-trip bit a bit + nunca inflar + tamanhos de referencia)
main.exe t

# 3. compilar os baselines Unishox2 e SMAZ (uma vez)
gcc -O2 -o tools/short/uni_cli.exe tools/short/uni_cli.c tools/short/unishox2.c
gcc -O2 -o tools/short/smaz_cli.exe tools/short/smaz_cli.c tools/short/smaz.c

# 4. (opcional) gerar um primer de dominio e recompilar para usar cp/dp embutido
python gen_primer.py --from-dataset datasets/.../loop_1.csv --skip-header
rustc +stable-x86_64-pc-windows-gnu -C opt-level=3 main.rs -o main.exe

# 5. validacao em dados reais. --quick (minutos) para sanidade; sem flag para o completo (~1 h)
python benchmark.py --quick
```

```
python benchmark.py          # gera resultados_reais.json e cross_domain.json

# 6. regenerar o artigo e esta documentacao a partir dos resultados
python gen_artigo.py
python gen_doc.py
```

Por usar sementes fixas e ferramentas de versão conhecida, a execução reproduz os números das tabelas da Seção 11 dentro da variação esperada de medição de tempo. O modo `--quick` grava em arquivos separados, para não sobrescrever os resultados canônicos.

10.6. Decisões metodológicas e seu porquê

Várias escolhas do protocolo existem para garantir rigor, e convém explicitá-las. A medição por mensagem, e não por arquivo inteiro, isola o efeito do prior na partida a frio, que é justamente o que o método corrige; medir arquivos diluiria esse efeito. A separação held-out, com o primer construído apenas a partir do treino, impede que a avaliação meça memorização em vez de generalização. A comparação contra a variante enxuta do `zstd` separa o ganho atribuível ao modelo do ganho atribuível ao formato sem cabeçalho, evitando creditar ao modelo uma vantagem que é, em parte, de enquadramento. A inclusão dos especialistas de string curta, `Unishox2` e `SMAZ`, situa o método ante seus concorrentes mais diretos no nicho, e não apenas ante compressores de uso geral. O experimento cruzado quantifica a dependência de domínio, que é a principal ameaça à validade. E o uso de mil mensagens por domínio, com a mediana e o percentil 95 além da média, garante que o resultado não reflita poucos casos extremos. Cada uma dessas decisões responde a uma pergunta que um avaliador atento faria.

11. Resultados

11.1. Tamanho comprimido por mensagem

A Tabela 2 traz o tamanho médio do comprimido, em bytes, por domínio e configuração, incluindo a variante enxuta do dicionário do `zstd` e os especialistas de string curta. Valores menores são melhores.

Tabela 2. Tamanho médio do comprimido por mensagem, em bytes.

Domínio	orig	GPA frio	GPA prior	unishox2	smaz	zstd dict	zstd lean	brofli
MQTT (IoT)	75,33	47,71	17,66	39,85	97,1	41,56	33,56	52
AIS (navios)	101,02	82,89	44,1	66,15	140,57	68,53	60,53	80,28
Sensores (Intel Lab)	63,48	48,74	26,08	39,21	91,02	51,87	43,87	47,63
Medidor (energia)	62,86	40,83	17,42	36,7	81,97	39,12	31,12	39,13
GPS (GeoLife)	64,95	51,3	23,93	37,39	88,26	48,62	40,62	47,05
Log Apache	83,51	75,14	10,1	58,23	81,75	31,96	23,96	78,9
Log Linux	106,89	90,29	14,9	71,57	105,55	36,04	28,04	96,38
Log OpenSSH	109,38	95,78	14,28	78,67	110,9	36,41	28,41	95,65
Log HDFS	141,76	114,49	32,12	94,78	151,92	53,29	45,29	117,53

SMS	80,63	66,63	38,99	52,72	57,48	63,91	55,91	67,01
Tweets	71,48	60,41	35,57	46,21	47,62	60,16	52,16	60,69

Em todos os domínios o GPA com prior produz o menor arquivo médio. Sobre o dicionário treinado do zstd, o fator de redução varia de 1,55 a 3,16 vezes, média 2,12. Esse número inclui o enquadramento do zstd. Contra a variante enxuta, que remove a soma de verificação e o identificador de dicionário, o fator cai para 1,37 a 2,37 vezes, média 1,73, valor que reflete apenas o modelo. Sobre o Unishox2, especialista sem prior, a média é 2,78 vezes.

11.2. Decomposição do ganho: formato e modelo

A diferença entre as duas comparações da seção anterior é o custo fixo de enquadramento que o zstd paga e o GPA não. Medido sobre uma entrada vazia, o dicionário do zstd ocupa 17 bytes na forma padrão e 9 bytes na forma enxuta. Em uma saída de poucas dezenas de bytes, isso é uma fração relevante. Parte da vantagem do GPA é, portanto, de formato, e decorre da ausência de cabeçalho; a parte restante, capturada pela comparação com a variante enxuta, é do modelo aquecido pelo prior. Ambas são propriedades legítimas do método, e a decomposição evita atribuir ao modelo um ganho que é, em parte, de formato. Reportar as duas comparações é a postura honesta diante da pergunta natural sobre por que a diferença é tão grande.

A Tabela 3 decompõe, em bytes médios por mensagem, a vantagem do GPA com prior sobre o dicionário do zstd. A coluna de moldura removível é a soma de verificação e o identificador de dicionário eliminados pela variante enxuta, da ordem de oito bytes constantes; a coluna de modelo e formato é o restante, que combina o modelo aquecido pelo prior e o cabeçalho irreduzível do zstd que o formato sem cabeçalho também evita.

Tabela 3. Decomposição da vantagem do GPA com prior sobre o zstd-dict (bytes por mensagem).

Domínio	Vantagem total	Moldura removível	Modelo + formato
MQTT (IoT)	23,9	8,0	15,9
AIS (navios)	24,4	8,0	16,4
Sensores (Intel Lab)	25,8	8,0	17,8
Medidor (energia)	21,7	8,0	13,7
GPS (GeoLife)	24,7	8,0	16,7
Log Apache	21,9	8,0	13,9
Log Linux	21,1	8,0	13,1
Log OpenSSH	22,1	8,0	14,1
Log HDFS	21,2	8,0	13,2
SMS	24,9	8,0	16,9
Tweets	24,6	8,0	16,6

11.3. Distribuição: mediana e percentil 95

A vantagem não se deve a poucos casos extremos. A Tabela 4 mostra que, para o GPA com prior, a mediana e o percentil 95 do tamanho por mensagem ficam próximos da média em todos os domínios, o que indica uma distribuição concentrada.

Tabela 4. GPA com prior: média, mediana e percentil 95 do tamanho (bytes).

Domínio	Média	Mediana	P95
MQTT (IoT)	17,66	17	22
AIS (navios)	44,1	45	55
Sensores (Intel Lab)	26,08	26	30
Medidor (energia)	17,42	17	21
GPS (GeoLife)	23,93	24	31
Log Apache	10,1	10	13
Log Linux	14,9	13	30
Log OpenSSH	14,28	13	23
Log HDFS	32,12	30	40
SMS	38,99	28	89
Tweets	35,57	33	66

11.4. Expansão e taxa de vitória

A Tabela 5 mostra a fração de mensagens que cada ferramenta expande e a fração em que o GPA com prior vence. Ela evidencia o problema que motiva o trabalho, pois no regime de poucas dezenas de bytes gzip e xz, e o SMAZ em dado estruturado, frequentemente aumentam o dado.

Tabela 5. Vitórias do GPA com prior e mensagens expandidas por outras ferramentas (%).

Domínio	GPA prior venc.	gzip exp.	xz exp.	smaz exp.	unishox2 exp.
MQTT (IoT)	100	54,4	100	100	0
AIS (navios)	98,9	88,1	100	100	0
Sensores (Intel Lab)	100	100	100	100	0
Medidor (energia)	100	97,6	100	100	0
GPS (GeoLife)	100	93,1	100	100	0
Log Apache	100	100	100	17,2	0
Log Linux	96,1	88,9	100	47,5	0
Log OpenSSH	99,2	79,7	100	61,9	0
Log HDFS	99,8	51	99,8	97,8	0
SMS	93,9	77,5	99,2	3,9	1,1
Tweets	95,3	81,6	100	2,2	0

11.5. Dependência do prior em relação ao domínio

A Tabela 6 quantifica o que ocorre quando o prior não casa com o domínio. Cada célula é o tamanho médio do comprimido ao usar o primer do domínio da coluna para comprimir as mensagens de teste do domínio da linha. A diagonal, com primer casado, é sempre a menor; fora dela o tamanho cresce

de forma acentuada e pode aproximar-se ou exceder o tamanho original, o que mostra que o ganho depende criticamente de um prior compatível com a aplicação.

Tabela 6. Tamanho médio (bytes) com primer cruzado: linha é o teste, coluna é o primer.

teste \ primer	MQTT	AIS	Sensores	GPS	Log
MQTT	18,18	46,33	56,75	46,9	52,41
AIS	91	43,21	92,16	78,25	95,91
Sensores	54,09	46,73	26,14	47,23	50,15
GPS	49,16	42,52	51,55	23,71	56,24
Log	99,14	97,29	102,03	103,2	9,82

11.6. Velocidade e memória

É preciso separar duas medidas distintas de recurso. A Tabela 7 traz a vazão e o pico de memória residente do processo no computador de teste, sobre lotes concatenados. Esse pico de processo inclui o binário e o sistema operacional e não representa o consumo do algoritmo embarcado; serve apenas para comparar o uso relativo de recursos entre as ferramentas no host, onde o GPA já usa muito menos que os compressores de janela longa.

Tabela 7. Vazão média e pico de memória do processo no host (não é o footprint embarcado).

Configuração	Vazão (MB/s)	Mem. proc. (MB)
GPA frio	7,0	7,1
GPA prior	7,2	9,3
gzip-9	9,8	6,6
zstd-19	10,0	51,9
brotli-11	3,1	20,5
xz-9	4,3	42,8

11.7. Footprint embarcado

O número relevante para microcontroladores não é o pico de processo, mas o conjunto de trabalho do motor, medido pelo alocador instrumentado e independente do tamanho da mensagem no perfil micro. A Tabela 8 reporta esse footprint. A ele soma-se o primer, que ocupa de 16 a 32 KB na memória de programa do dispositivo, pois reside no codec e não no arquivo; esse é um custo honesto a declarar, ainda que não trafegue em cada mensagem.

Tabela 8. Footprint de heap do motor (alocador instrumentado), fora o primer.

Cenário	Compressão	Descompressão
Perfil micro (até 4 KB)	cerca de 51 KB	cerca de 19 KB
Perfil de fluxo (64 KB)	cerca de 234 KB	cerca de 130 KB

No perfil micro, o decodificador, com cerca de 19 KB de heap mais o primer de até 32 KB, cabe com folga em microcontroladores das classes ESP32 e STM32, cuja memória é da ordem de centenas de quilobytes. A latência de compressão de uma mensagem de dezenas de bytes é da ordem de poucos microssegundos.

11.8. Exemplo concreto

A título de ilustração, uma mensagem MQTT real de 69 bytes, do tipo "1725866030.472465,0x0018, ... ,2,dos", é reduzida pelo GPA com prior a 16 bytes. As demais ferramentas, sem um prior casado, ficam bem acima: o Unishox2 produz 35 bytes, o zstd com dicionário 43 bytes, a variante enxuta 35 bytes, e o GPA sem prior 42 bytes.

Um exemplo de log esclarece por que os logs comprimem de forma tão agressiva. A linha do Apache "[Sun Dec 04 06:06:11 2005] [notice] workerEnv.init() ok /etc/httpd/conf/workers2.properties", de 91 bytes, é reduzida pelo GPA com prior a 10 bytes, ante 30 bytes do zstd com dicionário, 63 do Unishox2 e 82 do GPA sem prior. Essa linha é um molde quase fixo, repetido milhares de vezes no fluxo; o primer já o conhece, de modo que apenas a parte variável, sobretudo o instante de tempo, precisa ser codificada. A agressividade dos resultados de log, em que entradas de oitenta a cem bytes caem para dez a quinze, decorre da natureza altamente repetitiva desse tráfego, e não de um artefato de medição.

11.9. Comportamento fora do nicho

Em arquivos gerais grandes, a razão de compressão do GPA fica na classe do gzip, atrás de zstd, xz e brotli, em razão da janela de 32 KB e do modelo de baixa ordem. Em texto de linguagem natural livre, compressores de caractere bem ajustados permanecem competitivos. Em sequências de DNA, o modelo de ordem 2 é insuficiente e o método fica atrás até de ferramentas de uso geral. Esses regimes estão fora do propósito do GPA e são registrados para evitar leitura indevida dos resultados do nicho.

11.10. O trade-off de engenharia

Os experimentos anteriores estabelecem que o efeito existe, a Tabela 3 mostra de onde ele vem, e a Tabela 6 mostra onde ele deixa de valer. Reunidos, eles permitem enquadrar a proposta como uma decisão de engenharia, e não apenas como um número de compressão. A decisão troca um custo fixo por um ganho recorrente. O custo fixo é o primer, de 16 a 32 KB na memória de programa do dispositivo (Tabela 8), pago uma única vez na gravação do firmware e nunca transmitido. O ganho recorrente é a redução de cerca de duas a três vezes sobre o dicionário do zstd, ou de cerca de 1,7 vezes quando se isola o modelo do formato, em cada mensagem enviada.

O balanço depende de três grandezas: a memória de programa investida, a memória de trabalho em execução e os bytes economizados por mensagem. A memória de programa, de dezenas de kilobytes, é abundante nos microcontroladores usuais, cuja flash é da ordem de megabytes. A memória de trabalho do decodificador, de cerca de 19 KB, cabe com folga na RAM desses dispositivos. E a economia por mensagem se acumula a cada transmissão, de modo que, em um dispositivo que envia muitas mensagens curtas por enlaces onde cada byte custa energia e tempo de antena, o investimento fixo é rapidamente compensado. A condição para que o trade-off compense é a estabilidade do domínio: o experimento cruzado da Tabela 6 mostra que um primer descasado anula o ganho e pode inflar. O método é, assim, indicado quando o domínio é estável e o volume de mensagens é alto, e contraindicado quando o tráfego é heterogêneo ou a memória de programa é criticamente escassa. A pergunta útil sobre o trabalho não é se o ganho é real, e sim em que regimes esse trade-off entre flash, RAM e compressão se paga.

A Tabela 9 reúne, em um único lugar, os custos do prior, antes dispersos pelo texto, tornando o trade-off imediatamente visível.

Tabela 9. Custos do prior (resumo do trade-off).

Item	Valor
Primer (memória de programa)	16 a 32 KB
Decodificador (RAM de trabalho)	cerca de 19 KB
Compressor (RAM de trabalho)	cerca de 51 KB
Degradação com domínio cruzado	até cerca de 10 vezes (Tabela 6)

12. Implantação em hardware embarcado

Esta seção descreve como levar o GPA a um microcontrolador e, em especial, como validar o footprint sem o dispositivo físico, por compilação cruzada e análise estática. O foco é o ESP32, pela maturidade das ferramentas, mas o raciocínio vale para outras famílias como STM32 e nRF.

12.1. Alvo e arquitetura

O destino natural do método são dispositivos de recursos restritos, onde o ganho em bytes vira economia de energia e de tempo de antena. A arquitetura é assimétrica: o dispositivo comprime a mensagem antes de transmitir, com o codificador e o primer no firmware, e o gateway ou a nuvem descomprime; o sentido se inverte para o enlace de descida. Como o pipeline usa apenas aritmética inteira e é determinístico, não exige unidade de ponto flutuante e produz o mesmo resultado no microcontrolador e no servidor, o que é essencial para o casamento bit a bit entre os dois lados.

12.2. Adaptação do código

O motor de referência usa a biblioteca padrão do Rust. Para um microcontrolador há dois caminhos. O primeiro é portar o `ghost_core` para Rust no modo `no_std`, trocando `std::cmp::min` por `core::cmp::min` e fornecendo um alocador global, como o `esp-alloc`, para os poucos usos de `Vec` e `Box`; alternativamente, como os tamanhos por perfil são conhecidos em tempo de compilação, esses buffers podem virar vetores estáticos, dispensando alocação dinâmica. O segundo caminho é reimplementar o motor em C sobre o ESP-IDF, seguindo o guia do Apêndice C; é o caminho mais comum no ecossistema ESP32. Em ambos, o primer é embutido na memória de programa, por `include_bytes` no Rust ou por um vetor `const` no C, e o trabalho é feito sobre buffers em memória, sem entrada e saída de arquivo. O decodificador é o port mais simples, pois não aloca tabelas LZ: ele reconstrói a saída diretamente no buffer de destino.

12.3. Validação por compilação cruzada e análise estática

O ponto prático mais relevante é que boa parte do footprint pode ser validada no computador, sem o dispositivo. Basta compilar o firmware com a cadeia de destino e ler o mapa de memória do binário. No ESP-IDF, o fluxo é compilar e inspecionar o tamanho:

```
idf.py build      # compila o firmware (cross-compile para xtensa)
idf.py size      # imprime o mapa de memoria estatico do binario
```

A saída separa a memória de programa (flash) da memória estática (RAM), nas seções usuais de um binário embarcado, como ilustrado abaixo.

```
Flash:
.text      <codigo do motor: aritmetico + PPM + LZ>
.rodata    <constantes e o PRIMER (16 a 32 KB)>
RAM (estatica):
.data      <globais inicializados>
.bss       <buffers nao inicializados; o modelo, se alocado estaticamente>
```

Esses números validam, de forma reproduzível e sem hardware, a parte estática do footprint da Tabela 8: que o código e o primer cabem na flash e que as estruturas estáticas cabem na RAM. Um projeto que aloque o modelo em vetores estáticos, em vez de no heap, move o modelo para a seção .bss e torna o footprint completo diretamente visível nessa análise, sem depender de medição em tempo de execução. A Tabela 10 resume o que cada método de medição cobre.

Tabela 10. O que cada método de validação de footprint cobre.

Método	Onde	O que mede
Alocador instrumentado (bench)	PC	pico de heap dinâmico do motor (Tabela 8)
idf.py size (compilação cruzada)	PC	flash (.text e .rodata, com o primer) e RAM estática (.data e .bss)
esp_get_free_heap_size	dispositivo / QEMU	heap livre antes e depois, em execução
esp_timer_get_time	dispositivo / QEMU	latência por mensagem

Em resumo, a memória estática e o tamanho do código são verificáveis hoje, no PC, por compilação cruzada; restam para o dispositivo real, ou para o emulador QEMU que acompanha o ESP-IDF, a latência e o pico de heap dinâmico, além da medição de energia.

Essa validação deixou de ser apenas uma proposta. O repositório inclui, em embedded/, um decodificador de referência em C sem alocação dinâmica, com todo o estado em memória estática, portado de ghost_core.rs e empacotado como componente ESP-IDF. Ele foi verificado bit a bit contra o motor Rust nos vetores de conformidade e em mensagens reais, em modo normal e com prior, com doze de doze casos exatos. Compilado para o ESP32 com o ESP-IDF, o relatório idf.py size-components atribui ao componente do decodificador 19.692 bytes na seção .bss, 1.811 bytes de código em flash e 220 bytes de constantes, no alvo xtensa real. A Tabela 11 reúne as três medições independentes de RAM do decodificador, que convergem em torno de 19 KB e confirmam a linha da Tabela 8 no próprio hardware de destino.

Tabela 11. Footprint de RAM do decodificador por três métodos independentes.

Método	Onde	RAM do decodificador
Alocador instrumentado	motor Rust (PC)	19.083 B
size no objeto	C compilado no PC (x86)	19.776 B
idf.py size-components	ESP32 (xtensa)	19.692 B

O modo com prior, que adiciona as tabelas LZ do aquecimento, sobe para cerca de 259 KB de memória estática, ainda dentro da RAM de um ESP32. O firmware completo do exemplo, com todo o núcleo do ESP-IDF, ocupou cerca de 150 KB, deixando livre a maior parte da partição de aplicação.

12.4. Verificação de corretude no PC

Antes de levar o decoder ao dispositivo, convém confirmar no computador que ele reproduz exatamente a saída do motor Rust. O repositório traz um driver de teste, embedded/test/host_test.c, que lê um arquivo .gpa e o descomprime com o decoder em C. O procedimento é compilar o driver com o gcc e, para cada caso, comprimir com o main.exe e descomprimir com o binário em C, comparando byte a byte.

```
cd embedded/test
```

```
gcc -O2 -DGPA_ENABLE_PRIMED -I ../components/gpa_decoder/include \
  ../components/gpa_decoder/gpa_decoder.c host_test.c -o gpa_host.exe

# modo normal: comprime com o motor Rust, descomprime com o C, compara
main.exe c mensagem m.gpa && gpa_host m.gpa saida
# modo com prior: o mesmo primer dos dois lados
main.exe cpf mensagem m.gpa primer.bin && gpa_host m.gpa saida primer.bin
```

Nos vetores de conformidade e em mensagens reais de diferentes domínios, em modo normal e com prior, o decoder em C reproduziu a saída do motor de referência em doze de doze casos, bit a bit, incluindo o dado incompressível, em que o modo de armazenamento é exercido. É essa equivalência que autoriza usar o binário em C como decoder de produção no dispositivo: o formato não tem número de versão, então a compatibilidade entre as duas implementações depende da reprodução exata de cada modelo e do codificador, e a suíte de testes é a especificação operacional dessa compatibilidade.

12.5. Reprodução no ESP-IDF

Para medir o footprint no alvo e ver o decoder executar, usa-se o ESP-IDF. No Windows, a instalação é feita pelo instalador oficial, que traz o compilador xtensa, o idf.py e um Python próprio, e cria um atalho de terminal com o ambiente já ativado. A partir desse terminal, dentro de embedded, a medição do footprint é direta e não exige o dispositivo nem o emulador:

```
idf.py set-target esp32
idf.py build
idf.py size          # resumo de uso de memoria do firmware inteiro
idf.py size-components # contribuicao por componente; ver a linha gpa_decoder
```

Na medição usada nesta documentação, o componente gpa_decoder apareceu com 19.692 bytes de .bss, 1.811 bytes de .text em flash e 220 bytes de .rodata, no alvo xtensa, exatamente os valores da Tabela 11. Para executar o decoder no emulador, instala-se o QEMU da Espressif e roda-se o monitor:

```
python %IDF_PATH%\tools\idf_tools.py install qemu-xtensa # uma vez; reabrir o terminal
depois
idf.py qemu monitor # boot do ESP32 emulado; imprime o app_main; sair com Ctrl+]
```

O exemplo descomprime uma mensagem embutida, confere o round-trip bit a bit e imprime a latência, por esp_timer_get_time, e o heap livre antes e depois, por heap_caps_get_free_size. Como o decoder não aloca dinamicamente, o delta de heap é nulo, e todo o consumo aparece no .bss, já visível pelo idf.py size. Convém lembrar que a latência reportada pelo QEMU é indicativa, pois o emulador não é fiel ao ciclo do silício; a latência e a energia reais exigem a placa física, e ficam como medição futura sem alterar as conclusões de footprint.

12.6. Energia e airtime

O balanço energético reforça a viabilidade. Em enlaces de baixa potência, a transmissão de rádio domina o consumo, de modo que gastar alguns microssegundos de processador para reduzir o payload à metade prolonga a autonomia da bateria. Em tecnologias com limites rígidos de payload, como os doze bytes por quadro do Sigfox ou os tetos de ciclo de trabalho do LoRaWAN, a redução pode ser a diferença entre caber ou não em um quadro, ou entre respeitar ou não o limite regulatório. É nesse regime, de muitas mensagens curtas por enlaces caros, que o investimento fixo de flash do primer se paga.

13. Limites e ameaças à validade

A principal dependência do método é o casamento entre o prior e o domínio. Um primer genérico rende pouco quando o domínio de teste difere do corpus de treino, o que reforça a necessidade de

especializar o primer por aplicação. A velocidade, da ordem de poucos megabytes por segundo, é adequada a mensagens curtas mas limitante para grandes volumes. O modelo de ordem 2 sobre nibbles é deliberadamente simples, escolha que favorece a adaptação rápida em mensagens curtas mas limita o ganho em dados maiores. Por fim, a avaliação usa mil mensagens por domínio com sementes fixas; uma validação industrial deveria ampliar o número de amostras e de domínios.

14. Trabalhos futuros

Três direções se destacam. Um modelo de contexto de ordem mais alta, ou a mistura de modelos de várias ordens, para elevar a razão de compressão sem perder o baixo footprint. A medição em dispositivo real ou em emulador da latência e do pico de heap dinâmico, complementando a validação estática do footprint por compilação cruzada descrita na Seção 12. E um estudo do procedimento de construção do primer, incluindo seu tamanho ótimo e o efeito da diversidade do corpus sobre a generalização.

15. Conclusão

O GhostPredict mostra que um compressor especializado pode comprimir, de forma consistente, mensagens que os compressores de uso geral expandem. A combinação de formato sem cabeçalho, prior embutido no codec e garantia de não expansão entrega, no nicho de micro-payloads de IoT, o menor arquivo entre todos os concorrentes, incluindo especialistas de string curta e o dicionário treinado do zstd; isolando o modelo do formato, a vantagem sobre o dicionário enxuto do zstd permanece. Os ganhos observados dependem da existência de um prior compatível com o domínio da aplicação, condição que quantificamos pelo experimento cruzado e que define o escopo de uso do método. A especialização, e não a universalidade, é a tese e a força do projeto.

Referências

- Cleary, J. G. and Witten, I. H. (1984). Data compression using adaptive coding and partial string matching. *IEEE Transactions on Communications*, 32(4):396-402.
- Witten, I. H., Neal, R. M. and Cleary, J. G. (1987). Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520-540.
- Ziv, J. and Lempel, A. (1977). A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337-343.
- Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098-1101.
- Collet, Y. and Kucherawy, M. (2021). Zstandard Compression and the 'application/zstd' Media Type. RFC 8878, Internet Engineering Task Force.
- Alakuijala, J. and Szabadka, Z. (2016). Brotli Compressed Data Format. RFC 7932, IETF.
- Deutsch, P. (1996). DEFLATE Compressed Data Format Specification version 1.3. RFC 1951, IETF.
- Zheng, Y., Zhang, L., Xie, X. and Ma, W.-Y. (2009). Mining interesting locations and travel sequences from GPS trajectories. In *WWW 2009*, pages 791-800.
- Zhu, J., He, S., He, P., Liu, J. and Lyu, M. R. (2023). Loghub: a large collection of system log datasets for AI-driven log analytics. In *ISSRE 2023*.
- Go, A., Bhayani, R. and Huang, L. (2009). Twitter sentiment classification using distant supervision. Technical report, Stanford University.

Apêndice A. Constantes do sistema

Tabela A.1. Constantes principais do motor.

Constante	Valor	Papel
Alfabeto	18	16 nibbles + EOF + match
Codificador	32 bits	low, high inteiros; HALF = 2^{31}
Flag de modo	63 / 64	normal [0,63), armazenado [63,64)
Janela LZ micro / fluxo	4096 / 32768 B	perfil por tamanho de entrada
Hash LZ micro / fluxo	4099 / 16381	tabela de dispersão (primos)
Match mínimo / máximo	4 / 67	comprimento de referência
Profundidade de cadeia	128	candidatos por busca
Faixas de distância / comprimento	16 / 7	mais slots de referência
Limiar de reescalonamento	2048	halving das contagens
Bloco de fluxo	4 MB	RAM limitada no modo cs/ds

Tabela A.2. Faixas de distância e comprimento (base e bits extras).

Faixa	Base distância	Bits dist.	Base compr.	Bits compr.
0	1	0	4	0
1	2	0	5	0
2	3	1	6	1
3	5	2	8	2
4	9	3	12	3
5	17	4	20	4
6	33	5	36	5
7	65	6	-	-
8	129	7	-	-
9	257	8	-	-
10 a 15	513 a 16385	9 a 14	-	-

Apêndice B. Vetores de conformidade

Tabela B.1. Casos da suíte de conformidade e tamanhos de referência (modo normal).

Caso	Entrada	Tamanho .gpa
Entrada vazia	0 bytes	1 B
Literal único	'A'	2 B
Hello World	'Hello World!' (12 B)	13 B
Faixa completa	bytes 0 a 255 (256 B)	259 B
Repetição	'A' repetido 100 vezes	6 B

JSON IoT	registro de 37 B repetido 30 vezes	46 B
----------	------------------------------------	------

Os casos cobrem entrada vazia, literais isolados, dado incompressível, em que o modo de armazenamento limita a expansão a três bytes, repetição extrema e um payload estruturado do nicho. A igualdade exata com os tamanhos de referência protege contra regressões de formato.

Apêndice C. Guia de reimplementação

Para reimplementar o GPA em outra linguagem, basta portar o `ghost_core.rs`. A ordem sugerida e a lista de verificação são:

- Codificador e decodificador aritméticos inteiros de 32 bits, com tratamento de underflow por bits pendentes (Listagens 1 a 3). Validar com um teste de ida e volta de bits.
- IO de bits big-endian, idêntico nos dois lados (o bit de mais alta ordem é escrito primeiro).
- Modelo PPM de ordem 2 sobre nibbles, com escape método A e exclusão, e reescalonamento ao atingir 2048 (Listagens 4 e 5). O contexto avança só em nibbles.
- Buscador LZ77 por cadeias de hash de três bytes, com casamento preguiçoso e os limites de comprimento 4 e 67 (Listagens 7 e 8).
- Modelos de distância e comprimento, com recência por mover-para-frente e faixas com bits extras (Listagem 9 e Tabela A.2).
- Flag enviesada e modo de armazenamento, para a garantia de não expansão (Listagem 14).
- Aquecimento pelo primer e dicionário, idênticos nos dois lados, para o modo com prior (Listagens 12 e 13).

O critério de sucesso é a suíte de conformidade da Tabela B.1: round-trip exato e os mesmos tamanhos de referência. Como o formato não tem cabeçalho nem número de versão, a compatibilidade entre implementações depende da reprodução exata de cada modelo e do codificador, o que torna a suíte de conformidade a especificação operacional do formato.

Apêndice D. Ambiente e versões

Para reprodução fiel, a Tabela D.1 lista as ferramentas e versões utilizadas. As versões dos compressores afetam os tamanhos de saída; as do compilador e do ESP-IDF afetam o footprint. As sementes fixas dos scripts tornam os resultados estáveis entre execuções.

Tabela D.1. Ferramentas e versões.

Ferramenta	Versão / configuração	Uso
Rust (rustc)	stable, alvo x86_64-pc-windows-gnu	compilar o motor (o ligador MSVC falha)
gcc	MinGW-w64 / msys2	baselines Unishox2 e SMAZ; decoder C no host
zstd	1.5.7	baseline, com e sem dicionário; dicionário treinado por --train
xz	5.6.3	baseline
gzip	1.13	baseline
brotli	1.1.0	baseline
Unishox2 / SMAZ	fonte oficial (compilado)	especialistas de string curta
Python	3.11 (psutil; python-docx)	harness e geração dos documentos

ESP-IDF	v5.5.4	compilação para ESP32 e medição de footprint
Toolchain xtensa	xtensa-esp-elf 14.2.0	compilador cruzado do ESP-IDF
QEMU	qemu-xtensa (Espressif)	execução emulada do firmware

Apêndice E. Resolução de problemas comuns

Esta seção reúne percalços observados durante a reprodução e suas soluções.

Falha de ligação ao compilar o motor. o ligador MSVC falha neste projeto; use a cadeia GNU, com `rustc +stable-x86_64-pc-windows-gnu`.

Documentos não regeneram após instalar o ESP-IDF. o ESP-IDF coloca o seu próprio Python no caminho de busca, e nele a biblioteca `python-docx` não está instalada; invoque os scripts `gen_artigo.py` e `gen_doc.py` com o interpretador Python onde o `python-docx` está presente.

idf.py qemu acusa qemu-system-xtensa ausente. instale o emulador da Espressif com `python %IDF_PATH%\tools\idf_tools.py install qemu-xtensa` e reabra o terminal do ESP-IDF para que ele entre no caminho de busca.

Latência do QEMU diferente do esperado. o emulador não reproduz o ciclo exato do silício; a latência ali é indicativa. Para latência e energia reais, use a placa física.

zstd --train recusa por poucas amostras. treine o dicionário no modo de blocos, com a opção `-B` sobre um arquivo concatenado, em vez de um arquivo por amostra; foi o que o harness adotou.